



DANIEL ZHANG

Project 5: Creation of a CI/CD pipeline that automatically updates upon a push

A CI/CD pipeline was created to automate the deployment of the existing static website from Project 4. This pipeline integrates GitHub, AWS CodePipeline, and CodeBuild to automatically build and deploy changes to an S3 bucket whenever updates are pushed to the repository. The system builds on the previously established architecture (S3, CloudFront, Route 53, and ACM) by introducing automation and improving security through the use of a private S3 bucket and CloudFront Origin Access Control (OAC). As a result, the website can now be continuously updated with minimal manual intervention while ensuring that all content is securely delivered through the custom domain.

Services Used

Namecheap: Domain Registration

Namecheap was used to purchase a domain (zhangdaniel62csce412.me). This was used to have a human-readable domain name that users could use to access said website. It provides a readable and memorable address that users can enter in a browser.

Amazon S3: Hosting Service

Amazon S3 is a cloud storage service used to host static content such as HTML, CSS, and JavaScript files. It acts as a simple web server for static content. The direct endpoint to the S3 bucket was hidden, and the website is only accessible through the Cloudfront distribution. This is done to ensure that all traffic to the website goes through Cloudfront, which provides HTTPS and caching.

Amazon Route 53: DNS Service

Route 53 is used to route users that visit the domain to the correct AWS resource. This is important as it allows for traffic for the domain to the correct location. In addition, it is used for certificate validation through ACM. It translates the domain name into the appropriate Cloudfront distribution, allowing users to access the website using a human-readable URL.

AWS Certificate Manager (ACM): SSL Certificate

ACM was used to provision and deploy a secure HTTPS certificate. It is essential to convert the website from HTTP to HTTPS, which in essence encrypts the data between the user and the website.

Amazon Cloudfront: CDN + HTTPS Layer

Cloudfront is responsible for distributing the website hosted on S3 globally and enabling HTTPS using the ACM certificate that was provisioned by ACM. It caches content on edge servers (servers that are closest to the user) to improve latency and the general responsiveness of the website. This includes images, files, and stylesheets. In addition, Cloudfront is required because S3 does not natively support HTTPS for custom domains (such as the one that was purchased off of Namecheap). An origin access identity (OAI) was also created to restrict access to the S3 bucket, ensuring that all traffic goes through Cloudfront.

AWS CodeBuild: Build Service

CodeBuild is a fully managed build service that compiles source code, runs tests, and produces software packages that are ready to deploy. It was used to build the website and prepare it for deployment. In this case, it was used to sync the local files to the S3 bucket and create an invalidation for the Cloudfront distribution to ensure that the latest changes are reflected on the website.

AWS CodePipeline: CI/CD Service

CodePipeline is a fully managed continuous integration and continuous delivery (CI/CD) service that helps automate the build, test, and deploy phases of the release process. It was used to create a pipeline that automatically updates the website whenever a change is pushed to the main branch of the GitHub repository. The pipeline is triggered by a webhook from GitHub, which starts the build process in CodeBuild if a change is pushed. Once the build is complete, the changes are deployed to S3 and Cloudfront is flushed of its cache to reflect the latest changes on the website.

Together, these services allowed the website to be securely accessed through a custom domain (zhangdanie162csce412.me). The structure used is scalable and cost-efficient, as it does not require the management of physical servers.

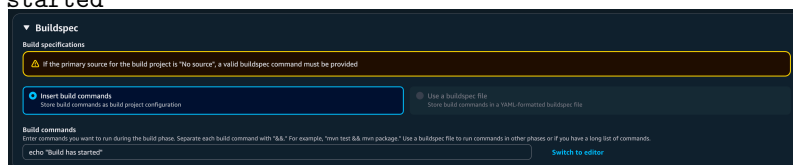
Process of the creation of the CI/CD pipeline

A CI/CD pipeline was created using AWS CodePipeline and CodeBuild to automate deployment of the static website. The pipeline pulls source code from a GitHub repository, runs a build step defined in a buildspec file, and deploys the updated files to an S3 bucket, with CloudFront invalidation to reflect changes immediately. The existing architecture from Project 4 (S3, CloudFront, Route 53, ACM) was extended by making the S3 bucket private and configuring CloudFront with Origin Access Control (OAC) so that content is only accessible through the domain. This allows any updates pushed to GitHub to automatically propagate to the live website while maintaining secure access.

Deploying an AWS CodeBuild build project:

1. Go to CodeBuild -> Build projects -> Create build project.
2. Name your project and select “Default Project” for the project type
3. Select “No source” as the source, as CodePipeline will provide the source.
4. Select the following in environment (should be the defaults):
 - provisioning model: on-demand
 - environment image: managed image
 - compute: EC2
 - running mode: container
 - OS: Amazon Linux
 - Runtime: Standard
 - Image version: use latest
 - rest are default
5. for Buildspec, choose “insert build commands”, and enter the following as a build command, which will be overridden later after the pipeline is fully built:

```
echo "Build has started"
```



6. Create the build project

Deploying CodePipeline:

1. Click on create new pipeline, then build custom pipeline:

The screenshot shows the 'Choose creation option' step (Step 1 of 7) in the AWS CodePipeline console. On the left, a sidebar lists steps from Step 1 to Step 7. The main area has a 'Category' section with three radio buttons: 'Deployment', 'Continuous integration', and 'Automation'. The 'Build custom pipeline' option is selected and highlighted with a blue border. At the bottom right, there are 'Cancel' and 'Next' buttons.

2. Name the pipeline, and change the execution mode to “Superseded”, which overrides a run if a new change is pushed to the website. Keep the following settings:

- Create new service role
- artifact store: Default location
- encryption key: Default AWS Managed Key

The screenshot shows the 'Choose pipeline settings' step (Step 2 of 7) in the AWS CodePipeline console. The left sidebar is the same as the previous screenshot. The main area is titled 'Pipeline settings' and includes several sections:

- Pipeline name:** A text input field containing 'csc-412-project-5-pipeline'.
- Execution mode:** Three radio buttons: 'Superseded' (selected), 'Queued', and 'Parallel'.
- Service role:** Two radio buttons: 'New service role' (selected) and 'Existing service role'.
- Role name:** A text input field containing 'AWSCodePipelineServiceRole-us-east-1-csc-412-project-5-pipeline'.
- Advanced settings:** A section with a dropdown arrow, containing:
 - Artifact store:** Two radio buttons: 'Default location' (selected) and 'Custom location'.
 - Encryption key:** Two radio buttons: 'Default AWS Managed Key' (selected) and 'Customer Managed Key'.
 - Variables:** A section with a 'Learn more' link, stating 'No variables defined at the pipeline level in this pipeline.' and an 'Add variable' button.

 At the bottom right, there are 'Cancel', 'Previous', and 'Next' buttons.

3. Select “GitHub (via GitHub App)” as the source provider, and connect a GitHub Connection to CodePipeline. If it isn’t created, go ahead and create one. Install an app to connect to the repository as a bot. Choose the correct repository as well as the branch which will have changes tracked. Ensure that “Start your pipeline on push and pull request events.” is enabled.

Add source stage info
Step 3 of 7

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via GitHub App)

Connection
Choose an existing connection that you have already configured, or create a new one and then return to this task.

arn:aws:codeconnections:us-east-1:123456789012:github:my-github-app or [Connect to GitHub](#)

Repository name
Choose a repository in your GitHub account.

zhangdaniel62/personal-website

You can type or paste the group path to any project that the provided credentials can access. Use the format 'group/subgroup/project'.

Default branch
Default branch will be used only when pipeline execution starts from a different source or manually started.

main

Output artifact format
Choose the output artifact format.

☒ **CodePipeline default**
AWS CodePipeline uses the default zip format for artifacts in the pipeline. Does not include Git metadata about the repository.

☐ **Full clone**
AWS CodePipeline passes metadata about the repository that allows subsequent actions to do a full Git clone. Only supported for AWS CodeBuild actions. [Learn more](#)

☒ **Enable automatic retry on stage failure**

4. Add the following webhook event filter to ensure that the pipeline is run only when a change is pushed directly to main or a PR is merged into the main branch:

- Event type: Push
- Filter type: Branch
- Branches or patterns: main

Webhook events

Webhook - optional

☒ Start your pipeline on push and pull request events.

Webhook event filters - optional [Remove filters](#)

Starts your pipeline on a specific event.

Event type
Choose the event type for the trigger that starts your pipeline.

Push

Start pipeline under these conditions - optional

Filter type **Branches or patterns**

Branch main

Use a comma to specify multiple entries.

File paths

^filepath.*

Don't start pipeline under these conditions - optional

[i](#) You can add additional sources and triggers by editing the pipeline after it is created.

5. Choose “Other build providers -> AWS CodeBuild” as the build provider, and select the CodeBuild build project made previously.

6. Select “Insert build commands”, and enter the following:

version: 0.2

phases:

pre_build:

commands:

- echo "Pipeline buildspec file ran"

build:

commands:

- aws s3 sync . s3://\$S3_BUCKET --delete --exclude ".git/*" --exclude "config/buildspec.yml"

- aws cloudfront create-invalidation --distribution-id \$DISTRIBUTION_ID --paths "/*"

7. Add the following environment variables as plaintext:

- Name of S3 bucket hosting the website
- ID of CloudFront distribution

8. Ensure that the following are correct:
9. Build type: Single build
10. Input artifact contains source artifact

[!IMPORTANT] In order to avoid potential issues, it is advised that the region of the pipeline is identical to the region that your S3 bucket is located

9. Skip the addition of a test phase, as it is not needed.
10. Skip the deploy stage, as it is not needed.
11. Ensure all information is correct, and create the pipeline.

[!NOTE] the parameter `DetectChanges` will appear to be false. This is expected, as a custom webhook was created.

Edits to S3 and CloudFront:

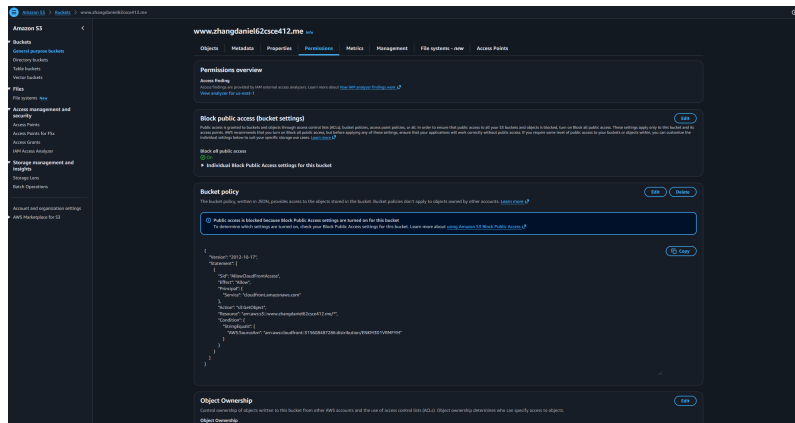
1. Enable Block public access (bucket settings) in S3 under permissions. All public access should be blocked.
2. Edit the bucket policy such that CloudFront is the only service allowed to directly access the S3 bucket and replace it with

```
{
  "Version": "2012-10-17",
  "Statement": [
```

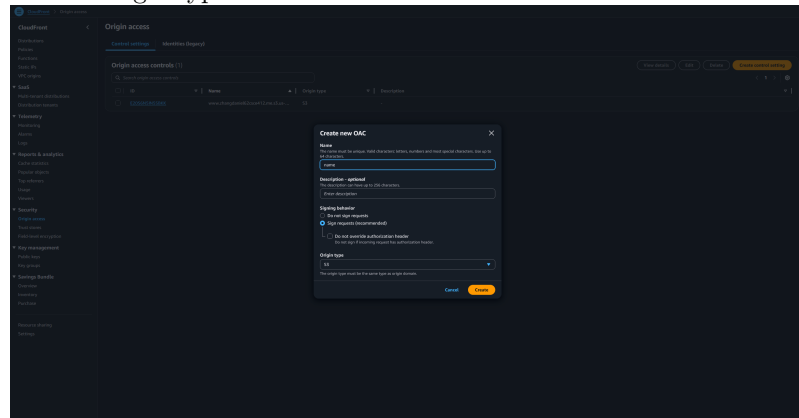
```

{
  "Sid": "AllowCloudFrontAccess",
  "Effect": "Allow",
  "Principal": {
    "Service": "cloudfront.amazonaws.com"
  },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::www.zhangdaniel62csce412.me/*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::315608487286:distribution/ENKM3D1VRMFYM"
    }
  }
}
]
}

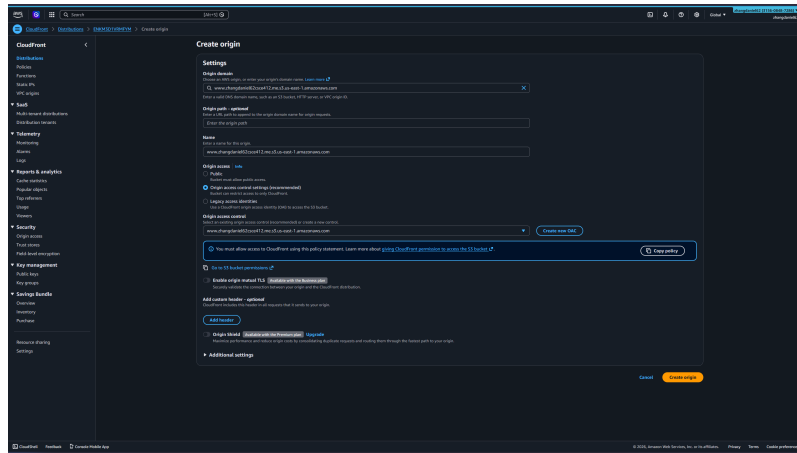
```



3. Head to CloudFront and enter the distribution that is used for the website
4. Under the “Security” tab, click on “origin access”. Create a new OAC, and make sure that “sign requests” is on, and the origin type is S3.



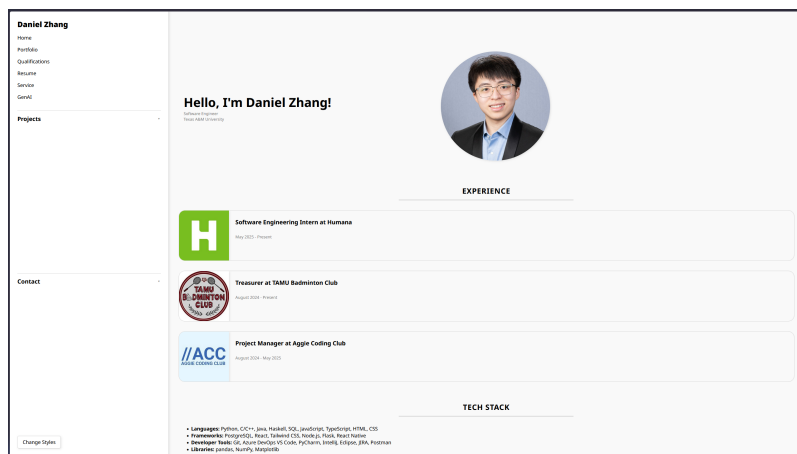
5. Go to the “Distributions” tab, and select the distribution that is used for the website. Click on “Edit” under the “Origins and Origin Groups” section.
6. Go to the Origin tab and click on “Create Origin”
7. Choose the correct origin domain as well as a name for the origin. Change origin access from “public” to “Origin Access Controls”. Select the OAC that was created earlier and create the origin.



How to update the website automatically

The website is automatically updated through the CI/CD pipeline whenever changes are pushed to the main branch of the GitHub repository. When a commit is pushed (or when a pull request is merged into main), CodePipeline is triggered via a webhook, which pulls the latest code, runs the build process in CodeBuild, and deploys the updated files to the S3 bucket. After deployment, a CloudFront invalidation is performed to ensure that cached content is refreshed and the latest version of the website is served. This allows updates to be reflected on the live site without any manual intervention.

Screenshot of the website before a change is pushed to GitHub, showing the old content:



Screenshot of the website after a change is pushed to GitHub, showing the new content:

